Rose's Documentation: Detecting Type 2 Diabetes from Chest X-rays with Pretrained Models (RAD-DINO)

Step 1: Finalize the dataset

a. **Clinical data -** `Clinical_Data_IncludedMissingSDI.csv` 1000 patients (317 T2D, 683 not).
b. **Images:** the **949 frontal images** (307 T2D and 642 not). The lateral images are excluded.
c. **Training/Testing split (**80% Training, 20% Testing):

	Images	T2D	Male	Average age
Train	759	32.3%	41.1%	54.2
Test	190	32.6%	41.1%	50.9

That way the test set represents the same population as the training set. (possibly looking at SDI for representing total population)

I created a script in the 'Useful python scripts' folder called make_train_test_folders.py so that I don't have to manually put the CXRs in their designated folders (testing or training). It is now easily automated

I created a script in the 'Useful python scripts' folder called haziness_ranking.py to most hazy CXRs

Model 1: all 949 images. Train 759, test 190, 32% T2D.

Model 2: lowest quality CXRs excluded (50) This leaves 899 images.

	Images	T2D	% T2D
Train	716	228	31.8%
Test	183	60	32.8%
Total	899	288	32.0%

Step 2: extract the RAD-DINO embeddings

I used google colab for this -> Extract the RAD-DINO embeddings.ipynb

1. **Model:** RAD-DINO, a vision transformer pretrained on chest X-rays. I used it as is, with no fine-tuning (the weights stayed frozen).

   (try fine tuning red dino)

2. **Input:** the 949 frontal CXRs in the final dataset (307 T2D, 642 not). Images were resized and normalized by the model's own preprocessing.
3. **Output:** each image became a list of 768 numbers (its "embedding"). I saved two versions: the CLS token, which is the model's summary of the whole image, and the average of the patch tokens. All embeddings were saved to `raddino_embeddings.npz`, and `master_table.csv` holds the labels, clinical data and train/test split in the same row order.
4. **Run once:** embeddings don't depend on the labels, so they were extracted a single time for all 949 images. Both runs reuse them: Run 1 uses all images, and Run 2 drops the 50 lowest-ranked images (the `exclude_lowest50` column).
5. **Environment:** Google Colab with a T4 GPU, in about 2 minutes.
6. **Checks:** 949 rows by 768 columns, in the same order as the master table, with no duplicates, missing values or all-zero rows.

Step 3: Create first few models (logistic regression)

I trained two image-only models, one for each way of summarizing an X-ray from Step 2. Each is a logistic regression that uses only the 768-number embedding and nothing from the spreadsheet.

- **CLS token:** RAD-DINO's own whole-image summary.
- **Patch-token average:** the average of the descriptions of all the small patches of the image.

Each model was trained on the 759 training images. The regularization strength was tuned with 5-fold cross-validation inside the training set.

How I scored them

- **AUC** means 0.5 is chance and 1.0 is perfect.
- **CV AUC** comes only from the 759 training images. I split them into 5 chunks, trained on 4, checked the model on the 1 it hadn't seen, rotated through all 5 combinations, and averaged the result. It was used to choose between models.
- **Test AUC** is the final exam. The 190 held-out images were never used in training or tuning. I scored each model once on them.
- **Accuracy, recall, precision and F1** use a 0.5 cutoff (predicted probability of 0.5 or higher counts as T2D).

Model	CV AUC	Test AUC	Accuracy	Recall	Precision	F1
CLS token	0.649	0.604	0.642	0.516	0.457	0.485
Patch-token average	**0.667**	**0.631**	0.611	0.532	0.423	0.471

I chose the **patch-token average** as the image-only model because it scored higher in cross-validation (in both runs). The two models are close, and the differences are within the uncertainty of a small test set

(the AUC 95% intervals are 0.52 to 0.69 for CLS and 0.55 to 0.71 for patch average). Both are only modestly better than chance.

Run	Model	Test images (T2D)	CV AUC	Test AUC	Accuracy	Recall	Precision	F1
Run 1: all 949	CLS token	190 (62)	0.649	0.604	0.642	0.516	0.457	0.485
Run 1: all 949	Patch-token average	190 (62)	**0.667**	**0.631**	0.611	0.532	0.423	0.471
Run 2: lowest 50 excluded	CLS token	183 (60)	0.623	0.600	0.607	0.450	0.409	0.429
Run 2: lowest 50 excluded	Patch-token average	183 (60)	**0.656**	**0.632**	0.628	0.550	0.446	0.493